

01 Vectors



1.4 Vector-Scalar Multiplication

Scaling vectors and spanning space with linear combinations

Linear Algebra Made Easy and Visual | Open-source code & drafts: <https://github.com/Visualize-ML>

What You Will Learn

1

Grasp the algebraic definition: multiplying $\mathbf{a}$ by scalar k scales every component to give $k\mathbf{a}$

2

Understand geometrically how scalar multiplication stretches a vector: positive keeps direction, negative flips it, zero collapses it

3

Understand two special values: $k=-1$ gives the opposite vector, $k=0$ gives the zero vector

4

Master the linear combination $\mathbf{c}=k\mathbf{a}+t\mathbf{b}$, a weighted sum that builds a new vector

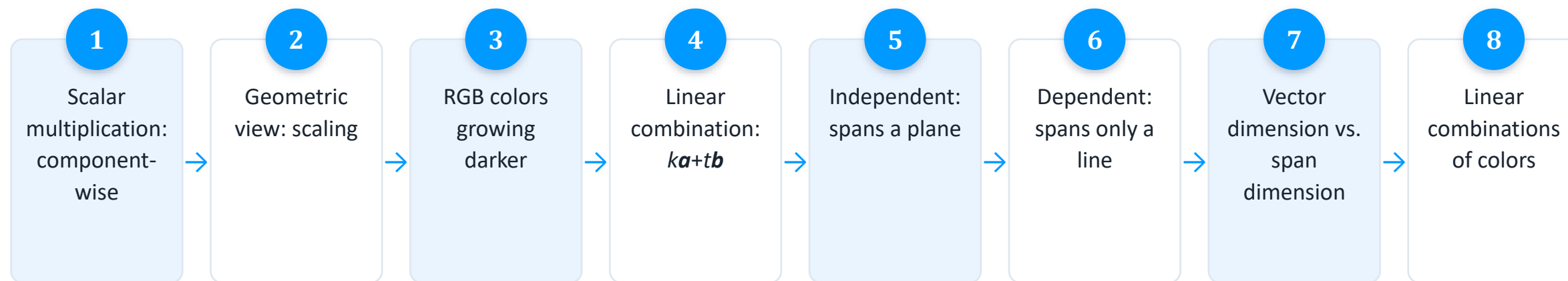
5

Distinguish linear independence (not collinear, spans a full plane) from linear dependence (collinear, spans only a line)

6

Implement scalar multiplication and linear combinations, and visualize them, with NumPy and Matplotlib

How the Ideas Connect



Scalar Multiplication: Component by Component

$$k\mathbf{a} = k[a_1; a_2; \cdots; a_n] = [ka_1; ka_2; \cdots; ka_n]$$

- Given a nonzero n-D column vector $\mathbf{a}$ and a scalar k , multiplication scales every component
- The result is again an n-D vector; both its direction and length can change with k
- $a_1, a_2, \dots, a_n$ are the scalar components of $\mathbf{a}$ (unknowns shown in italic)
- Next, a one-line NumPy check: `k * a_vec`

Scalar Multiplication with NumPy

*np.linalg.norm() measures length; $k * a_vec$ scales it*

- Define a 1-D array `a_vec` with `np.array()` — this represents the vector
- Compute the length (magnitude, L2 norm) of `a_vec` with `np.linalg.norm()`
- NumPy multiplies a scalar and an array directly: `k * a_vec` scales every component
- Call `np.linalg.norm()` again on `k_a_vec` and compare the new length

```
## 初始化
import numpy as np

## 定义向量
a) a_vec = np.array([3, 4])
b) np.linalg.norm(a_vec)

## 标量乘法
k = -2
c) k_a_vec = k * a_vec
d) np.linalg.norm(k_a_vec)
```



Scalar Multiplication Is a Scaling Transformation

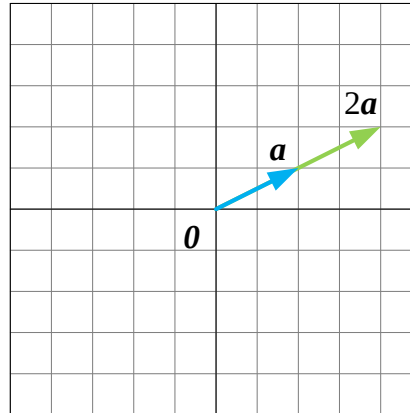
$|k| > 1$ stretches the vector; $0 < |k| < 1$ shrinks it; $k < 0$ flips its direction

- A positive scalar stretches or shrinks a vector along the same direction; a negative scalar flips it
- The new vector $k\mathbf{a}$ is always collinear with the original nonzero vector $\mathbf{a}$: same line, same or opposite direction
- Special case: $k=-1$ makes $k\mathbf{a}$ the opposite vector of $\mathbf{a}$ — equal length, opposite direction, symmetric about the origin
- Special case: $k=0$ makes $k\mathbf{a}$ the zero vector $\mathbf{0}$

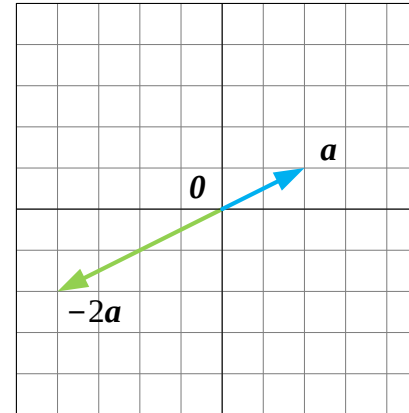
Figure 1 · Scaling a Vector

- (a)(b) $|k| > 1$ stretches the vector: $k=2$ keeps direction, $k=-2$ flips and stretches
- (c)(d) $0 < |k| < 1$ shrinks the vector; (e) $k=-1$ gives the opposite vector; (f) $k=0$ gives the zero vector

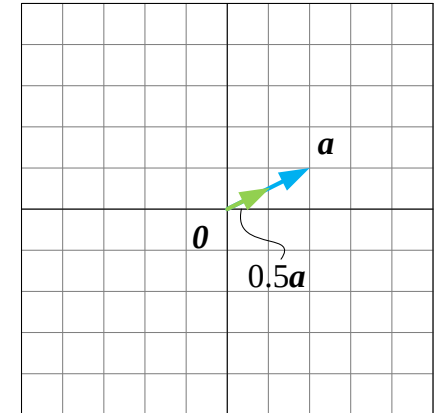
(a), $k = 2$



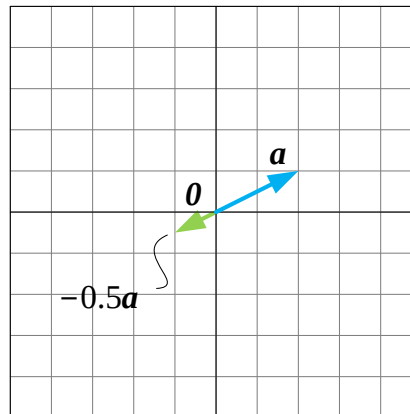
(b), $k = -2$



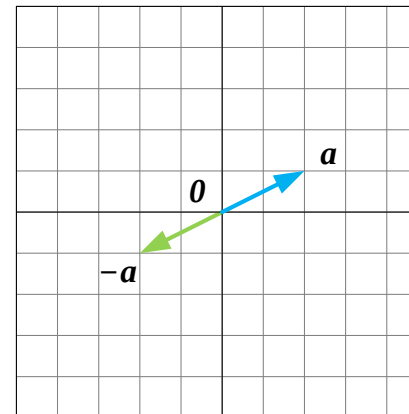
(c), $k = 0.5$



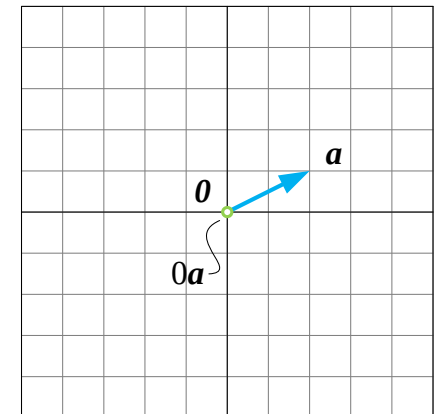
(d), $k = -0.5$



(e), $k = -1$



(f), $k = 0$



Visualizing Scalar Multiplication

matplotlib.pyplot.arrow() draws an arrow for each vector

- `plt.arrow()` needs a start point, an end point, and arrowhead style parameters
- 0, 0 is the arrow's start; `a_vec[0]` and `a_vec[1]` give its x- and y-components
- `head_width` and `head_length` control the arrowhead's size for clarity
- `fc` and `ec` set the arrow's fill and edge color; `label` supplies the legend text
- In hex color codes, `'#FF0000'` is pure red, `'#00FF00'` is pure green, `'#0000FF'` is pure blue

```
## 初始化
import numpy as np
import matplotlib.pyplot as plt

## 定义向量
a_vec = np.array([2, 1])

## 标量乘法
k = -2
k_a_vec = k * a_vec

## 可视化
plt.figure(figsize=(6,6))

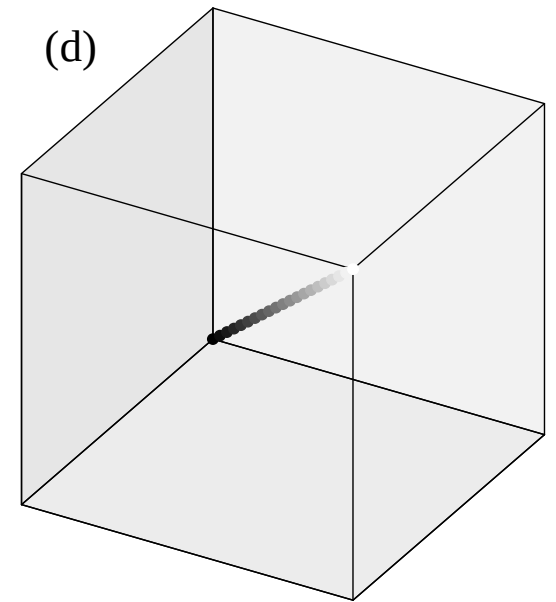
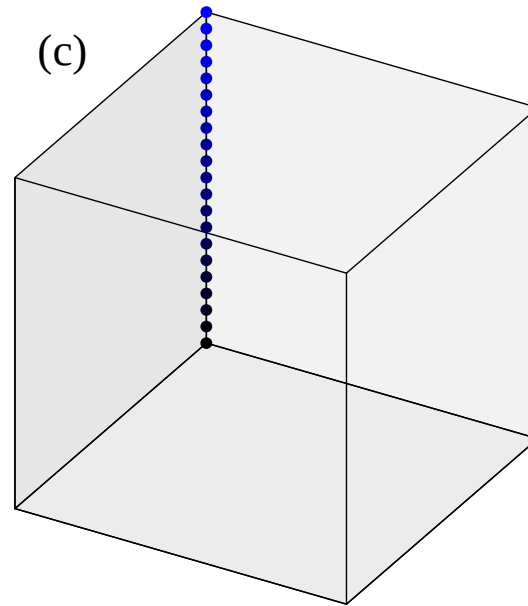
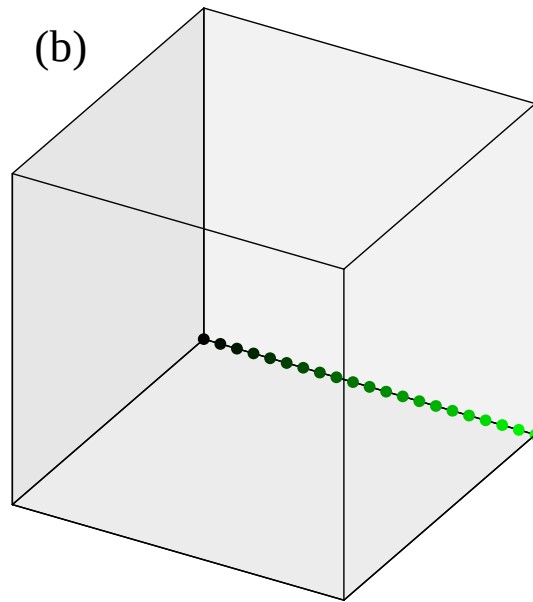
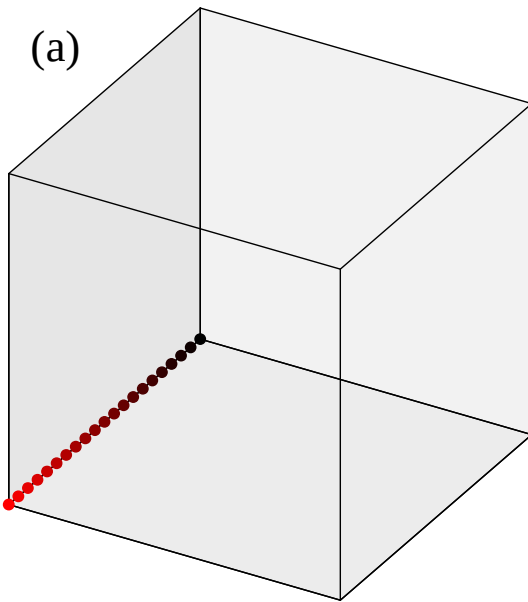
# 绘制向量 a
plt.arrow(0, 0, a_vec[0], a_vec[1],
          head_width=0.2, head_length=0.3,
          fc='#00B0F0', ec='#00B0F0', label="a")

# 绘制向量 k * a
plt.arrow(0, 0, k_a_vec[0], k_a_vec[1],
          head_width=0.2, head_length=0.3,
          fc='#92D050', ec='#92D050', label="k*a")

# 装饰
plt.gca().set_aspect('equal', adjustable='box')
plt.xlim(-5, 5) # 设置 x 轴范围
plt.ylim(-5, 5) # 设置 y 轴范围
plt.axhline(0, color='black', linewidth=0.5)
plt.axvline(0, color='black', linewidth=0.5)
plt.xticks(np.arange(-5,6))
plt.yticks(np.arange(-5,6))
plt.grid(color='gray', alpha=0.8, linestyle='-', linewidth=0.25)
plt.legend()
```

Figure 2 · Red, Green, Blue, and White Growing Darker

- The red vector $\mathbf{e}_1$, green vector $\mathbf{e}_2$, and blue vector $\mathbf{e}_3$ are each scaled by k ($0 < k < 1$)
- As k approaches 0, each vector shortens and its color darkens — but the direction (hue) never changes



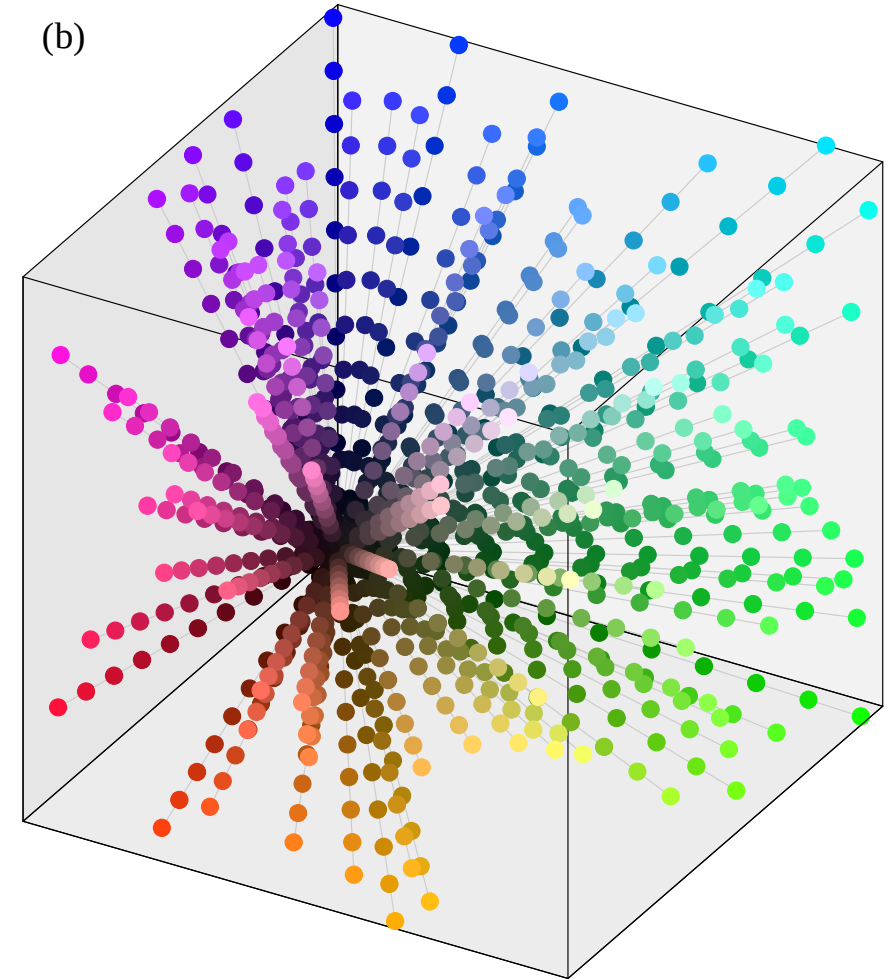
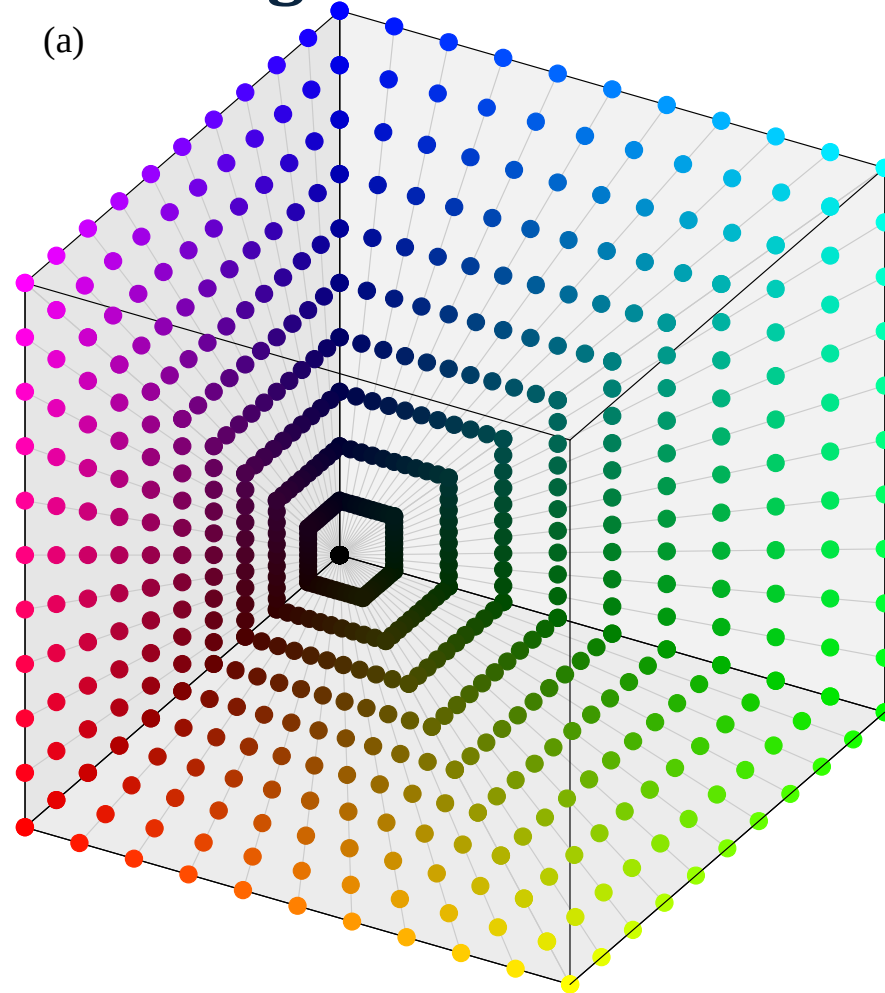
Scaling the All-Ones Vector: Shades of Gray

$$k(\mathbf{e}_1 + \mathbf{e}_2 + \mathbf{e}_3) = k[1;1;1] = [k;k;k]$$

- The 3-D all-ones vector $[1,1,1]^T$ is white; scaling it by k ($0 < k < 1$) produces gray
- As k approaches 1 the gray lightens toward white; as k approaches 0 it darkens toward black
- $k=0$ gives black and $k=1$ gives white — sweeping k smoothly interpolates from black to white

Figure 3 · Colors Growing Darker

- (a)(b) two panels define a set of colors in different ways and plot the corresponding scatter sequence
- Each sequence darkens through scalar multiplication: every base color is scaled by a continuously varying k ($0 < k < 1$)



Linear Combination: Scaling Plus Addition

$$\mathbf{c} = k\mathbf{a} + t\mathbf{b}, \text{ where } k, t \text{ are real numbers}$$

- Given two nonzero vectors $\mathbf{a}$, $\mathbf{b}$ of the same dimension, weight and sum them with scalars k , t
- A linear combination is a new vector built from a weighted sum of a set of vectors and their scalar coefficients
- Geometrically, varying k and t traces out a line through the origin, or fills an entire plane

Linear Independence: Non-Collinear Vectors Span a Plane

No scalar k satisfies $\mathbf{a} = k\mathbf{b} \Leftrightarrow \mathbf{a}, \mathbf{b}$ are linearly independent

- If two nonzero vectors $\mathbf{a}, \mathbf{b}$ are not collinear, they are called linearly independent
- Neither can be expressed as a scalar multiple of the other, so geometrically they never sit on the same line
- Linearly independent $\mathbf{a}, \mathbf{b}$ span a 2-D plane, written $\text{span}(\mathbf{a}, \mathbf{b})$
- Span is the set of all linear combinations of a group of vectors — everything addition and scaling can generate

Figure 4 · $\mathbf{a}$ and $\mathbf{b}$ Weave a Planar Grid (2-D Vectors)

- (a)–(d) in all four panels $\mathbf{a}$ and $\mathbf{b}$ are non-collinear, i.e. linearly independent
- Over all real k, t , $\mathbf{a}$ and $\mathbf{b}$ span the whole plane; over integers, $k\mathbf{a}+t\mathbf{b}$ lands on a regular lattice

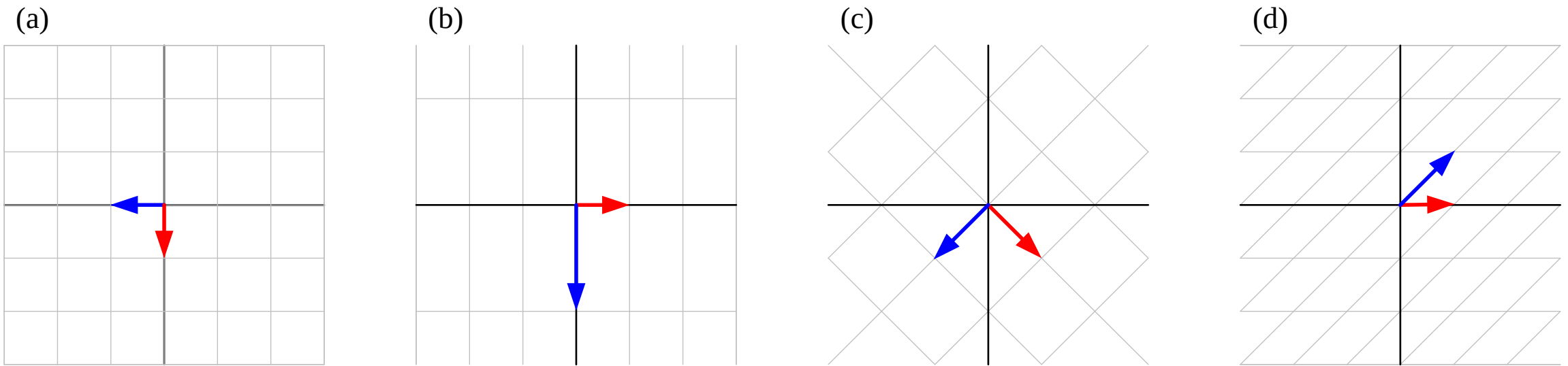
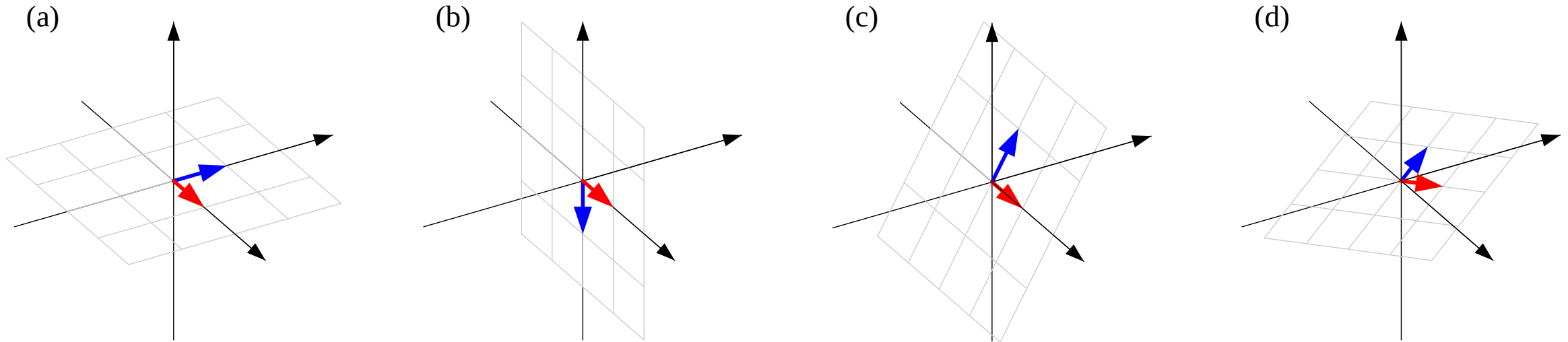


Figure 5 · a and b Weave a Planar Grid (3-D Vectors)

- Two non-collinear (linearly independent) 3-D vectors $\mathbf{a}$, $\mathbf{b}$ still span a 2-D plane through the origin
- Linear independence means neither vector is a scalar multiple of the other — each contributes its own direction



Linear Dependence: Collinear Vectors Span Only a Line

Some scalar k satisfies $\mathbf{a} = k\mathbf{b} \Leftrightarrow \mathbf{a}, \mathbf{b}$ are linearly dependent

- When $\mathbf{a}$ and $\mathbf{b}$ are collinear, they are called linearly dependent
- Every linear combination of $\mathbf{a}$ and $\mathbf{b}$ stays on that one line through the origin — it never fills a higher-dimensional space
- Intuitively, $\mathbf{b}$ is just some multiple of $\mathbf{a}$, pointing the same or opposite way, adding no independent direction

Figure 6 · $\mathbf{a}$ and $\mathbf{b}$ Are Collinear (2-D Nonzero Vectors)

- (a)–(d) in all four panels $\mathbf{a}$ and $\mathbf{b}$ are collinear, i.e. linearly dependent
- They span only a line through the origin — no choice of k, t can reach the rest of the plane

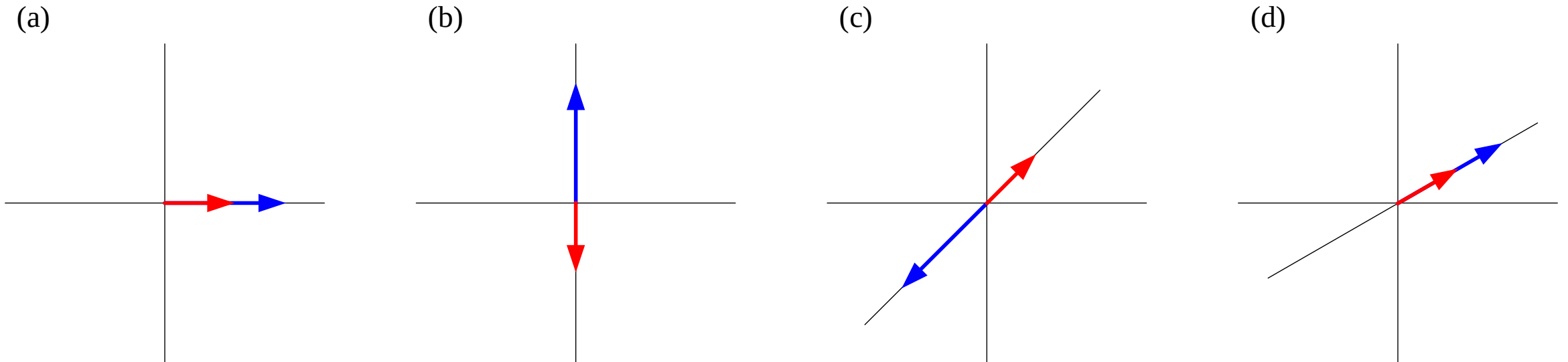
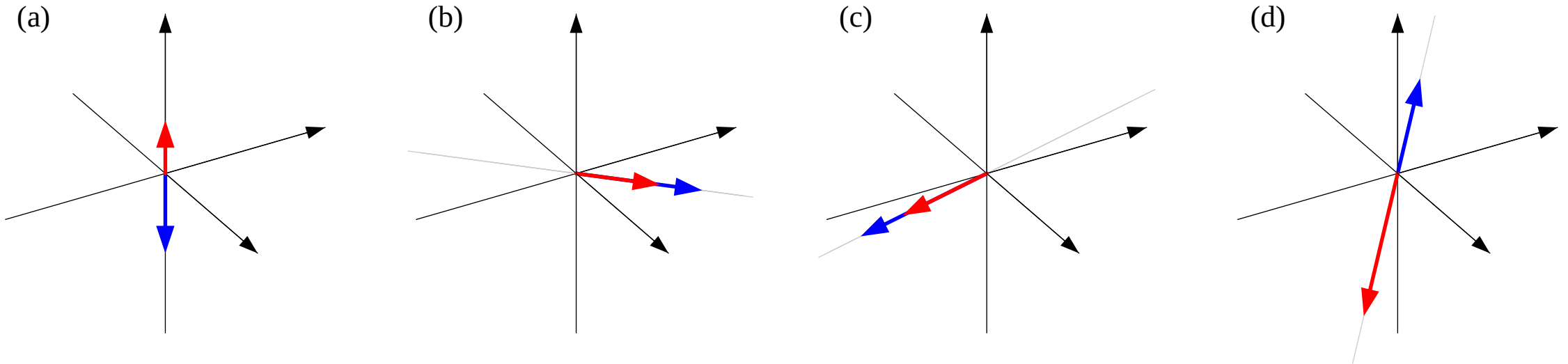


Figure 7 · $\mathbf{a}$ and $\mathbf{b}$ Are Collinear (3-D Nonzero Vectors)

- In 3-D space, if nonzero vectors $\mathbf{a}$, $\mathbf{b}$ are linearly dependent, their combinations still lie on one line through the origin
- However k , t are adjusted, $k\mathbf{a}+t\mathbf{b}$ stays confined to that line and can never span a plane



A Vector's Dimension Is Not the Dimension It Spans

$\mathbf{x} = [x_1, x_2]^T$ needs a 2-D plane to draw, but $\text{span}(\mathbf{x})$ is only a (through-origin) line

- A 2-D nonzero vector needs a 2-D plane to be drawn, yet the space it spans is only "1-dimensional"
- However you scale it up or down, it stays on the same line (or its reverse) — it never sweeps out the whole plane
- The same holds for a 3-D nonzero vector: it needs 3-D space to visualize, but its scalar multiples still trace just a line
- A vector's dimension is how many components it has; the dimension it spans is how much space those multiples cover

Linear Combinations in RGB Color Space

$$k_1 \mathbf{e}_1 + k_2 \mathbf{e}_2 = k_1 [1; 0; 0] + k_2 [0; 1; 0]$$

- In RGB color space, a linear combination is simply a mix of colors
- For pure red $\mathbf{e}_1$ and pure green $\mathbf{e}_2$, the random coefficients k_1, k_2 follow a $[0,1]$ continuous uniform distribution
- As k_1, k_2 range over all real numbers, $k_1 \mathbf{e}_1 + k_2 \mathbf{e}_2$ sweeps out the entire $x_1 x_2$ plane

Figure 8 · Linear Combinations of Two Colors

- (a) pure red e_1 combined with pure green e_2 ; (b) pure red e_1 combined with pure blue e_3
- (c) pure green e_2 combined with pure blue e_3 (left for you to analyze), with coefficients uniform on $[0,1]$

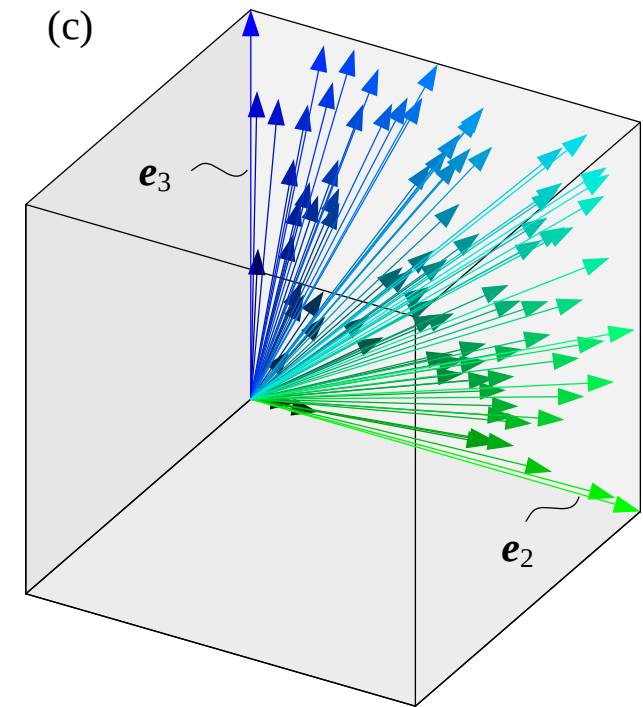
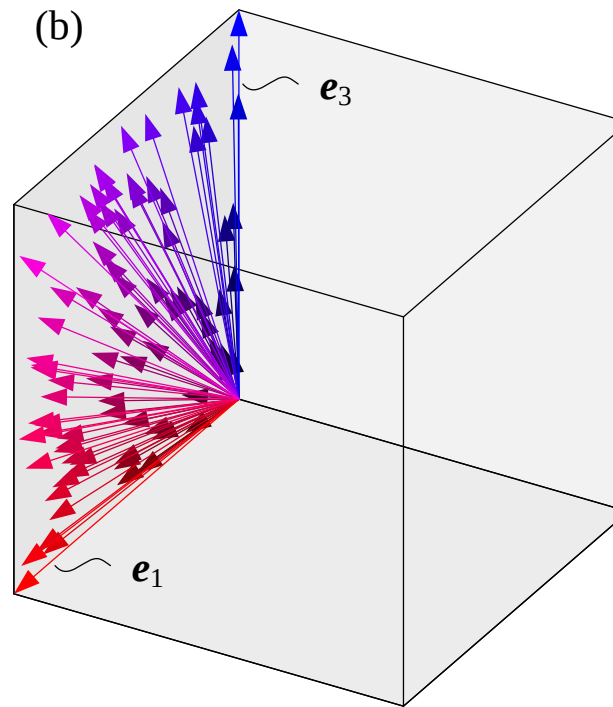
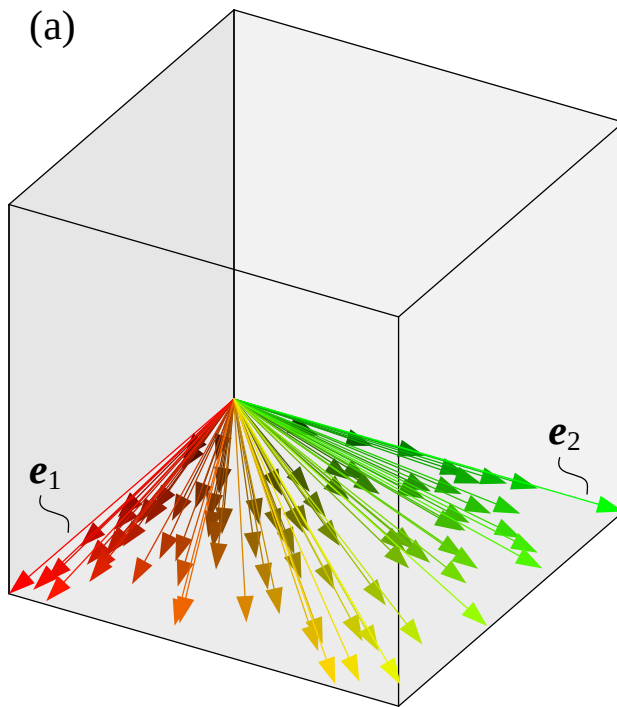
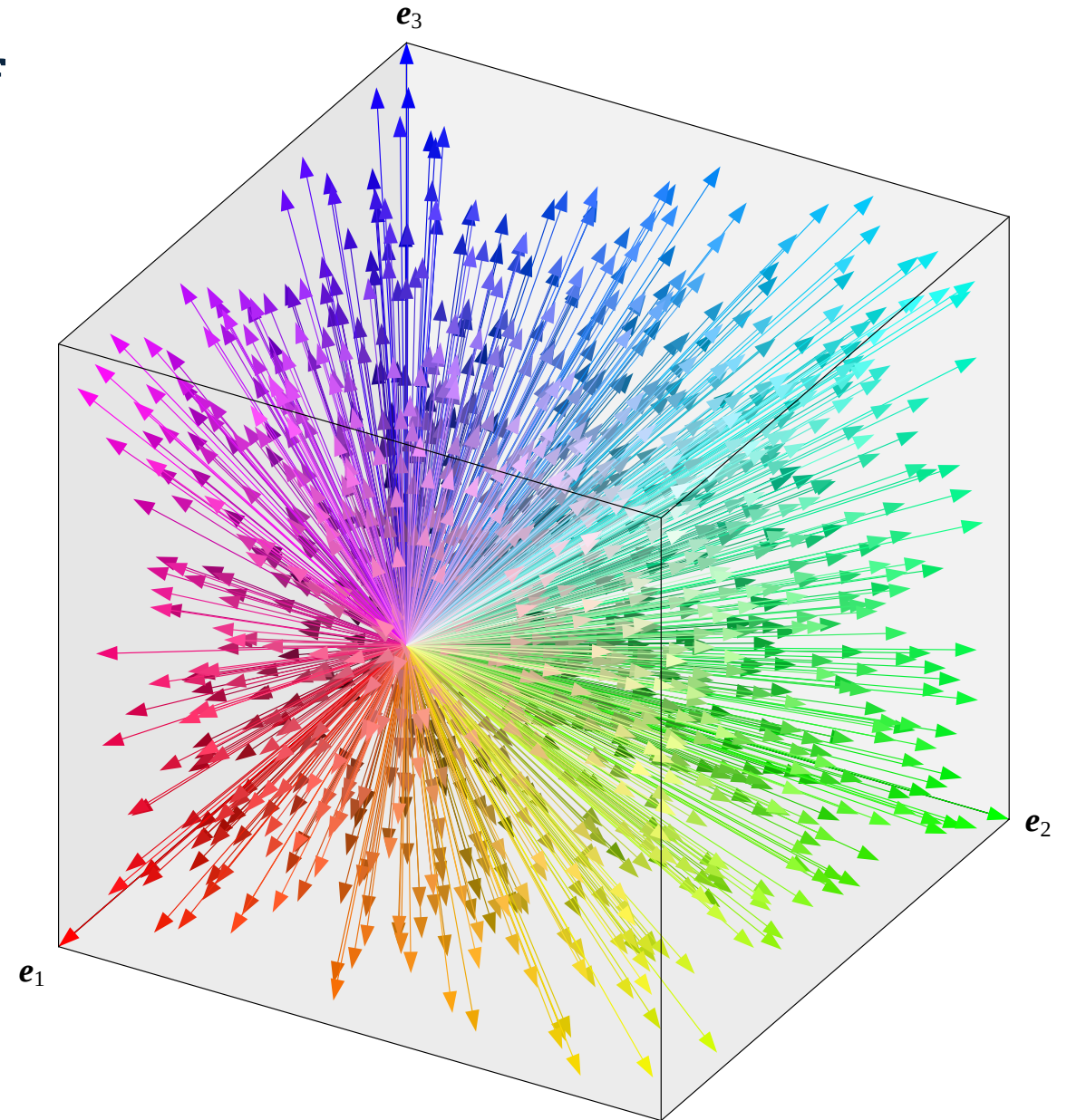


Figure 9 · A Linear Combination of Three Color Vectors

- Weighted mixing of pure red $\mathbf{e}_1$, pure green $\mathbf{e}_2$, and pure blue $\mathbf{e}_3$ fills out the colorful RGB space
- Coefficients k_1, k_2, k_3 are each independently drawn from a $[0,1]$ uniform distribution
- Three linearly independent basis vectors, combined linearly, span the entire 3-D color space



Key Terms in This Section

Scalar Multiplication $k\mathbf{a} = k[a_1; \dots; a_n]$

Zero Vector $k=0$ gives $k\mathbf{a} = \mathbf{0}$

Linearly Independent *not collinear — spans a whole plane*

Span *the set of all linear combinations of a group of vectors*

Vector Dimension *component count — distinct from the dimension it spans*

Opposite Vector $k=-1$ gives $k\mathbf{a} = -\mathbf{a}$

Linear Combination $\mathbf{c} = k\mathbf{a} + t\mathbf{b}$

Linearly Dependent *collinear — spans only a 1-D line*

Collinear *two vectors lying on the same (through-origin) line*

Continuous Uniform Distribution *every point in an interval equally likely*

FURTHER LEARNING

Companion Code & Extended Reading

Code 1 · LA_01_04_01.ipynb

Scalar multiplication and vector length with NumPy

Code 2 · LA_01_04_02.ipynb

Visualizing scalar multiplication with matplotlib.pyplot.arrow()

Open-Source Repository

Full code and figure assets: <https://github.com/Visualize-ML>

Q1 – Q9

Q1 $\mathbf{a}, \mathbf{b}, \mathbf{c}$ are 2-D nonzero vectors; if they are linearly dependent, what does that look like geometrically?

Q3 Mix pure red $[1,0,0]^T$ with dark red $[1/2,0,0]^T$ — what characterizes the resulting color?

Q5 Write Python code to reproduce the scatter plot in Figure 3(a).

Q7 Which of these vector pairs can "span" the whole plane: $[1,0]^T, [0,1]^T$ / $[1,2]^T, [2,1]^T$ / $[1,2]^T, [1,2]^T$ / $[1,0]^T, [-1,0]^T$?

Q9 Use NumPy to verify $(k+t)\mathbf{a} = k\mathbf{a} + t\mathbf{a}$, scalar addition distributing over a vector.

Q2 $\mathbf{a}, \mathbf{b}, \mathbf{c}$ are 3-D nonzero vectors; if they are linearly dependent, what does that look like geometrically?

Q4 Mix white $[1,1,1]^T$ with gray $[1/2,1/2,1/2]^T$ — what characterizes the resulting color?

Q6 Write Python code to reproduce the arrow plot in Figure 9.

Q8 Use NumPy to verify the distributive law $k(\mathbf{a} + \mathbf{b}) = k\mathbf{a} + k\mathbf{b}$.

Q10 – Q18

Q10 Write `scale_vector(a, k)` returning $k\mathbf{a}$, and check with `np.linalg.norm()` that the new length is $|k|$ times the original.

Q12 Write code that tests whether scalar k flips the direction ($k < 0$) or changes the length ($|k| \neq 1$).

Q14 Write `gray_scale(k)` returning the color $k \cdot [1, 1, 1]^T$, and show the color swatch with Matplotlib.

Q16 Use NumPy to test whether two 3-D vectors are linearly dependent (hint: cross product = zero vector).

Q18 Randomly generate 100 pairs of 2-D vectors and report the fraction that are linearly dependent (collinear).

Q11 Plot the same vector $\mathbf{a}$ for $k = -2, -1, -0.5, 0, 0.5, 1, 2$ with Matplotlib and observe the pattern.

Q13 Draw 100 random k in $[0, 1]$, scale pure red $\mathbf{e}_r$ by each, and plot the resulting color gradient bar.

Q15 Use NumPy to test whether two 2-D vectors are linearly dependent (hint: cross product / determinant = 0).

Q17 Write `span_plane(a, b, k_range, t_range)` to generate the grid of $k\mathbf{a} + t\mathbf{b}$ points and plot it (mimic Figure 4).

Q19 – Q27

Q19 Plot two linearly dependent vectors $\mathbf{a}$, $\mathbf{b}$ and their combination $k\mathbf{a}+t\mathbf{b}$; verify it always lands on the same line.

Q21 Use `mpl_toolkits.mplot3d` to plot a 3-D vector $\mathbf{a}$ and its scalar multiple $k\mathbf{a}$, and verify collinearity.

Q23 Generate random RGB color vectors with NumPy, scale each by $k=0.3$, and plot a before/after comparison.

Q25 Write `opposite_vector(a)` returning the opposite of $\mathbf{a}$, and plot $\mathbf{a}$ together with its opposite in Matplotlib.

Q27 Write code verifying that linear combinations of $\mathbf{e}_1$, $\mathbf{e}_2$, $\mathbf{e}_3$ cover the first octant of the RGB color cube.

Q20 Given vector $\mathbf{a}$, plot $k\mathbf{a}$ for $k=0.2, 0.4, 0.6, 0.8, 1.0$ in different colors to mimic the gradient in Figure 3.

Q22 Write `is_independent(a, b)` returning whether two vectors are linearly independent.

Q24 Verify in Python that when $k=0$, $k\mathbf{a}$ is the zero vector regardless of what $\mathbf{a}$ is.

Q26 Generate $k_1\mathbf{e}_1+k_2\mathbf{e}_2$ with k_1, k_2 drawn uniformly from $[0,1]$, repeat 500 times, and plot the scatter (mimic Figure 8).

10 Prompts to Explore Further (1/2)

- 1 Explain why the sign of scalar k (positive/negative) decides whether a vector's direction flips, while $|k|$ decides how its length changes.
- 2 If scalar multiplication in RGB color reads as "darkening," is there a vector operation that reads as "lightening"?
- 3 Prove that a nonzero vector $\mathbf{a}$ and any nonzero scalar multiple $k\mathbf{a}$ are always collinear — why is this the strict definition of "collinear"?
- 4 Using an everyday example, explain the intuitive difference between "linearly independent" and "linearly dependent" (say, two people pulling a box in different directions).
- 5 Why does a 2-D or 3-D vector always span only a "1-dimensional" space by itself? How does that differ from the dimension of the space it lives in?

10 Prompts to Explore Further (2/2)

6 Design a small Python experiment verifying that $k(\mathbf{a}+\mathbf{b})=k\mathbf{a}+k\mathbf{b}$ holds for arbitrary vectors.

7 Explain the concept of "span" and why two linearly independent 2-D vectors can span the entire plane.

8 If three 3-D vectors are linearly dependent, what is the highest dimension their linear combinations can span? Illustrate with a figure.

9 Reflect on what operation in color space corresponds to "darkening" via scalar multiplication — how would you achieve "lightening" with a vector operation?

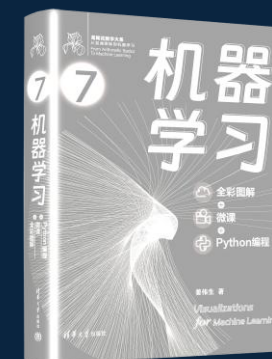
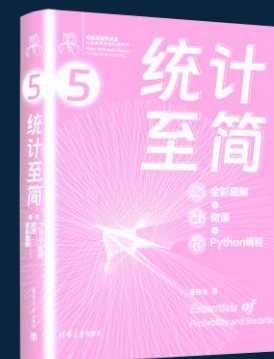
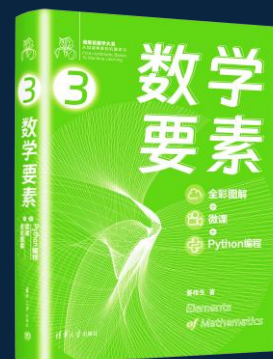
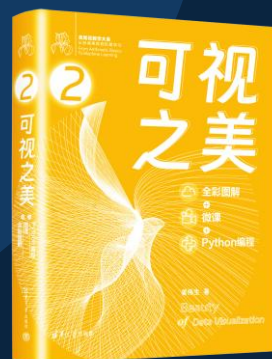
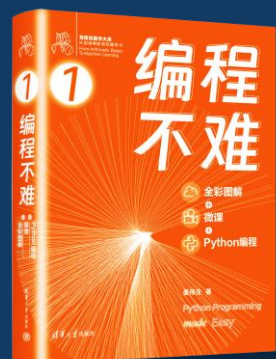
10 Use a concrete scenario (velocity, force, or color mixing) to explain the real-world meaning of scalar multiplication and linear combination.

数学

数学基础

线性代数

概率统计



Python编程

数据可视化

工具

数据分析
图和网络

回归、降维
分类、聚类

实践

